

Section 6: Table / Index Access Path

1. 인덱스의 본질

- 인덱스는 테이블과 별개의 저장 구조다.
 - 조회 성능을 높여주지만, **INSERT / UPDATE / DELETE** 시 함께 유지·관리 비용이 든다.
 - 따라서 인덱스는 많을수록 좋은 것이 아니라, 필요한 곳에 정확히 써야 한다.
-

2. 가장 중요한 인덱스 개념

B-Tree Index

- Oracle에서 기본적으로 생성되는 인덱스다.
- 리프 블록에는 보통 다음이 들어 있다.
 - 인덱스 키 값
 - ROWID
- 리프 블록끼리 **양방향 연결(double-linked)** 되어 있어서 **범위 조회(Index Range Scan)**에 유리하다.
- 가장 중요한 판단 기준은 **선택도(selectivity)**다.
 - 중복이 적고
 - 원하는 행을 잘 좁혀낼 수 있으면
 - 인덱스 효율이 좋다.

Function-Based Index

- 컬럼 원값이 아니라 **함수 적용 결과값**을 인덱스에 저장한다.
- `UPPER(col)`, `TRUNC(date_col)` 같은 형태로 조회가 자주 일어날 때 고려한다.

Bitmap Index

- **선택도가 낮은 컬럼**에서 고려할 수 있다.
- 값을 0/1 비트맵 형태로 매우 압축해서 표현한다.
- 보통 OLTP보다는 **분석/집계성 환경**에서 더 적합하다.

IOT(Index Organized Table)

- 특정 컬럼만 자주 읽는 경우,

테이블 본문으로 다시 가지 않고 인덱스 구조 자체에서 바로 읽도록 최적화한 형태다.

3. Access Path의 핵심

- **Access Path** = 쿼리가 데이터를 가져오는 방식
- 크게 두 가지다.
 1. **Table Access Path**
 2. **Index Access Path**

Oracle은 **비용 기반 최적화(CBO)** 를 사용하므로,
결국 **비용이 가장 낮다고 판단한 실행계획**을 선택한다.

4. Table Access Path 핵심

Full Table Scan

테이블 전체를 읽는 방식이다.

다음 경우에 자주 선택된다.

- 적절한 인덱스가 없을 때
- 조건의 선택도가 낮을 때
- 테이블이 매우 작을 때
- 읽어야 할 데이터 양이 많을 때

핵심 포인트:

- Full Table Scan은 무조건 나쁜 것이 아니다.
- 같은 테이블 데이터가 물리적으로 모여 있어
대량 읽기에는 오히려 효율적일 수 있다.

Table Access by ROWID

- 인덱스에서 얻은 **ROWID**로 실제 테이블 행을 읽는 방식
- 인덱스를 사용한 뒤 실제 데이터를 가져올 때 매우 자주 등장한다.

핵심 포인트:

- 인덱스는 보통 값 + ROWID 를 찾는다.
 - 그 다음 실제 데이터는 ROWID로 테이블에 접근해서 가져온다.
-

5. Index Access Path 핵심

1) Index Unique Scan

- PK / Unique Key 를 = 조건으로 정확히 찾을 때 사용
- 결과가 1건임이 보장될 때 가장 효율적이다.

2) Index Range Scan

- 범위 조건 (>, <, BETWEEN, 'ABC%') 에서 자주 사용
- 실행계획에서 가장 자주 보는 인덱스 접근 방식 중 하나다.

핵심 포인트:

- 인덱스는 정렬 구조이므로
시작점부터 끝점까지 범위를 훑는 방식이 가능하다.

3) Index Full Scan

- 인덱스를 순차적으로 끝까지 읽는 방식
- 특히 다음 상황에서 중요하다.
 - ORDER BY
 - GROUP BY
- 인덱스 정렬 순서를 활용할 수 있어 정렬 비용을 줄일 수 있다.

4) Index Fast Full Scan

- 필요한 컬럼이 이미 인덱스 안에 전부 있을 때
- 테이블까지 가지 않고 인덱스만 읽어서 결과를 반환할 수 있다.
- 여러 블록을 비순차적으로 동시에 읽어 대개 더 빠르다.

핵심 포인트:

- Full Scan은 순차적
- Fast Full Scan은 다중 블록 비순차 읽기

- 대신 Fast Full Scan은 정렬 순서를 보장하지 못하므로
ORDER BY , GROUP BY 최적화에는 불리할 수 있다.

5) Index Skip Scan

- 복합 인덱스에서 선두 컬럼 없이 뒤쪽 컬럼 조건만으로 인덱스를 활용하는 방식
- 자주 쓰지 않는 조건 때문에 별도 인덱스를 추가하기보다,
기존 복합 인덱스를 재활용하는 전략이다.

핵심 포인트:

- “원래 이 컬럼 전용 인덱스는 아닌데, 우연히 기존 복합 인덱스 안에 포함돼 있어서 활용하는 방식”

6) Index Join Scan

- 하나의 인덱스로는 부족하지만,
여러 인덱스를 조합하면 필요한 컬럼을 모두 충족할 수 있을 때
테이블 대신 인덱스들만 조합해 결과를 만드는 방식이다.

핵심 포인트:

- **SELECT 절의 모든 컬럼이 인덱스 안에 있어야 한다.**
- 가능하면 테이블 접근을 피하고 성능을 높일 수 있다.

6. 실무적으로 가장 중요한 원칙

1) 인덱스가 있다고 무조건 좋은 게 아니다

- 조회는 빨라질 수 있지만
- DML 비용은 증가한다.
- 항상 조회 이득 vs 유지 비용을 같이 봐야 한다.

2) 선택도가 중요하다

- 중복이 적고 원하는 행을 잘 줄여주는 컬럼이 인덱스에 유리하다.

3) 필요한 컬럼만 조회해야 한다

- 인덱스 안에 필요한 컬럼이 모두 있으면
테이블 접근 자체를 생략할 수 있다.

- 그래서 `SELECT *` 는 성능상 불리할 수 있다.

4) ORDER BY / GROUP BY 는 인덱스와 매우 밀접하다

- 정렬/그룹핑 대상이 **인덱스 컬럼과 잘 맞으면**
추가 SORT 비용을 줄일 수 있다.
- 특히 Index Full Scan이 강력해질 수 있다.

5) 실행계획은 “인덱스를 탔나 안 탔나”만 보면 안 된다

정말 봐야 할 것은:

- Full Table Scan인지
- ROWID Access인지
- Range Scan인지
- Full Scan인지
- Fast Full Scan인지
- Skip Scan인지
- Join Scan인지

즉, 어떤 **access path**를 탔는지가 핵심이다.

Oracle 튜닝의 핵심은 “인덱스를 만들었는가”가 아니라, 옵티마이저가 어떤 **Access Path**를 왜 선택했는지를 이해하는 것이다.

7. 적용 포인트

Bad SQL 우선 개선 방향

실제로 가장 자주 실행되거나 리소스를 가장 많이 사용하는 **SQL**부터 우선 개선하는 것이 효과적이다.

DB 모니터링 툴이나 **통계성 SQL**을 활용해 Bad SQL을 추출

이렇게 추려낸 SQL 중에서 우선순위를 정한 뒤

실행계획을 확인하여 왜 느린지 원인을 분석

현재 조회 SQL에서 불필요한 컬럼까지 모두 가져오는 경우가 많고,
이 때문에 인덱스를 타더라도 결국 **TABLE ACCESS BY ROWID** 나 **FULL TABLE SCAN** 이 발생할 수 있다.

- 정말 필요한 컬럼만 조회하고 있는가
- WHERE 조건 컬럼이 적절한 인덱스를 타는가
- 인덱스만으로 결과를 반환할 수 있는가
- 실행계획에서 **TABLE ACCESS BY ROWID** 를 줄이거나 없앨 수 있는가

만약 조회에 필요한 컬럼이 모두 인덱스 안에 포함되면,
옵티마이저는 테이블까지 가지 않고 **Index Fast Full Scan** 으로 처리.
이 경우 테이블 접근이 줄어들어 I/O 감소와 응답속도 개선을 기대

1. 모니터링 툴/통계 쿼리로 Bad SQL 추출
2. 사용량·자원소모 기준으로 우선순위 선정
3. 실행계획 확인
4. 필요한 컬럼만 조회하도록 수정
5. 인덱스만으로 처리 가능한 구조인지 검토
6. 개선 전/후 성능 비교